ITC 6050 — Data Engineering

Final Team Project Brief

Spring Term 2026 | Graduate School | MS in Data Science
Instructor: Dr. Maira Kotsovoulou
Submission Deadline: 21 July 2026 | **Presentations:** 21 July 2026

---

1. Project Overview

Each group will design, build, and present a complete **end-to-end data pipeline** using a real-world public API. The project demonstrates your mastery of the full data engineering stack covered in this course.

Groups of **2–3 students** will be assigned one of six project topics (see Section 6). All projects share the same technical requirements, grading rubric, and deliverables — only the data domain and API differ.

---

2. Group Allocations

Submissions via **Blackboard**. Each group submits one report (PDF) and one GitHub repository link.

Group	Size	Project	API Authentication
Group 1	3	Project 1 — Global Air Quality Monitor	None required
Group 2	3	Project 2 — Movie Industry Analytics	TMDB API key (free)
Group 3	3	Project 3 — Cryptocurrency Market Pipeline	None required
Group 4	3	Project 4 — Open Library Book Trends	None required
Group 5	3	Project 5 — GitHub Open Source Pulse	GitHub Personal Access Token (free)
Group 6	2	Project 6 — World University Rankings Pipeline	None required

> **Note for Groups 2 & 5:** Register for your API key/token as soon as groups are announced — both are free but require account setup before you can start ingesting data.

---

3. Required Technology Stack

API Key Registration

Some projects require an API key. **All keys are free** — no payment or subscription is needed.

Project	API Key Required	How to Register
Project 1 — Air Quality	No	No registration needed
Project 2 — Movies	**Yes**	Create a free account at themoviedb.org, then go to Settings → API to generate your key
Project 3 — Cryptocurrency	No	No registration needed
Project 4 — Open Library	No	No registration needed
Project 5 — GitHub	**Yes**	Log in to GitHub → Settings → Developer Settings → Personal Access Tokens → Generate new token (select `public_repo` scope)
Project 6 — Universities	No	No registration needed

> **Important:** Never commit your API key or token to GitHub. Store it in a `.env` file and add `.env` to your `.gitignore`. Use `.env.example` with placeholder values for submission.

Required Technology Stack

All projects must use the following tools. **No substitutions are permitted** without prior written approval.

Layer	Tool	Purpose
Ingestion	**dlt**	Extract data from API and load into PostgreSQL
Storage	**PostgreSQL**	Raw and analytics schema storage
Transformation	**dbt**	Staging model + mart model + tests
Quality	**dbt tests**	`not_null`, `unique`, `accepted_values`, custom tests
Dashboard	**Streamlit**	Interactive visualisation with filters
Version Control	**Git + GitHub**	All code committed with meaningful history
Environment	**Python venv**	Isolated dependencies, `requirements.txt` included

---

4. Deliverables & Grading

The project is worth **55% of your final grade** and consists of three deliverables:

Deliverable	Weight	Submission Format
Technical Report (3,000 words)	40%	PDF via Blackboard — 21 July 2026

Deliverable	Weight	Submission Format
Code Repository	30%	GitHub link submitted via Blackboard — 21 July 2026
Live Presentation (15 minutes)	30%	In-class, 21 July 2026

3.1 Technical Report

The report must be **2,800–3,200 words** (excluding code snippets and figure captions). It must contain the following sections **in order**:

#	Section	Words	Key Content
1	Introduction	~300	Problem statement, real-world relevance, who uses this data
2	Data Source & API	~300	API description, fields ingested, rate limits, data quality notes
3	Pipeline Architecture	~400	Architecture diagram, tool choices and justification, ETL vs ELT
4	Data Modeling	~400	ERD, raw schema, staging model, mart model, normalization decisions
5	Transformation Logic	~400	dbt model SQL explained, business logic, calculated fields
6	Data Quality	~300	dbt tests implemented, what they check, test results
7	Dashboard & Visualisation	~400	Screenshots, chart rationale, filter design, intended user
8	Challenges & Lessons Learned	~300	Real problems encountered, how you solved them, what you would do differently
9	Conclusion	~200	Summary, production readiness, future enhancements

3.2 Code Repository

Your GitHub repository must contain the following structure:

```
itc6050-project-groupX/
├── pipeline.py              ← dlt ingestion script
├── dashboard.py             ← Streamlit dashboard
├── requirements.txt         ← all dependencies
├── README.md                ← setup instructions
├── .gitignore               ← includes .env, .venv, *.duckdb
├── .env.example             ← template (no real keys)
└── analytics/               ← dbt project
    ├── models/
    │   ├── sources.yml
    │   ├── schema.yml
```

```
│   ├── stg_[topic].sql
│   └── [mart_model].sql
└── dbt_project.yml
```

3.3 Presentation (15 minutes)

The presentation consists of **12 minutes of demo + slides** followed by **3 minutes of Q&A**. Both group members must speak.

Slide	Content	Time
1	Project title, group members, topic	1 min
2	Data source: what API, what data, why it matters	2 min
3	Architecture diagram: walk through each layer	2 min
4	**Live demo:** run `pipeline.py` → `dbt run` → Streamlit	4 min
5	Data quality: show dbt test results	1 min
6	Challenges and lessons learned	1 min
Q&A	Instructor questions to each group member individually	3 min

> **Warning:** Q&A questions will be directed to **individual members**. Both members must be able to explain any part of the code.

---

5. Grading Rubric

Criterion	Marks	What we look for
Report — Architecture & Justification	15	Clear diagram, tool choices explained with reasoning, ETL vs ELT addressed
Report — Data Modeling	10	ERD present, staging vs mart explained, normalization decisions justified
Report — Quality & Transformation	10	dbt tests explained, transformation logic clear, business rules documented
Report — Challenges & Reflection	5	Specific real problems described with genuine reflection (not generic)
Code — Pipeline (dlt)	15	Runs without errors, paginates API, adds repo/source column, no credentials committed
Code — dbt Models & Tests	10	Both models present, at least 3 dbt tests, `dbt run` and `dbt test` pass
Code — Git History & README	5	Meaningful commit messages, README includes setup instructions, `.gitignore` correct

Criterion	Marks	What we look for
Presentation — Live Demo	15	Pipeline runs live, dashboard loads with real data, filters work
Presentation — Q&A	10	Both members can explain code, answer architecture questions, discuss trade-offs
Presentation — Clarity & Structure	5	Clear narrative, time kept, slides support the demo (not replace it)
TOTAL	**100**	

6. Project Assignments

Each group is assigned one project below. The technical requirements (stack, structure, number of models, number of tests) are **identical across all projects**.

Project 1 — Global Air Quality Monitor

Domain: Environmental Science

API	OpenAQ (`api.openaq.org`)
Authentication	None required
Locations endpoint	`https://api.openaq.org/v2/locations`
Measurements endpoint	`https://api.openaq.org/v2/measurements`
Docs	https://docs.openaq.org

Description: Ingest real-time air quality measurements (PM2.5, NO2, CO, O3) from cities worldwide. Build a pipeline that tracks pollution trends over time and flags cities exceeding WHO safety thresholds.

Required Raw Tables

Table	Key Columns
`measurements`	`location_id`, `parameter`, `value`, `unit`, `date_utc`, `city`, `country`
`locations`	`id`, `name`, `city`, `country`, `lat`, `lon`, `is_active`

Required dbt Models

- **Staging:** `stg_air_quality` — clean nulls, cast `date_utc` to timestamp, filter invalid readings (`value < 0`)
- **Mart:** `city_daily_avg` — group by city + parameter + date, compute daily average and max value

Required dbt Tests (minimum 3)

- `not_null` on `measurement_id`, `city`, `parameter`, `value`

- `accepted_values` on `parameter`: `['pm25','no2','co','o3','so2']`
- Custom test: `value >= 0` (no negative readings)

Required Dashboard Components

- KPI row: total cities monitored, total readings loaded, date range
- Bar chart: top 10 most polluted cities (avg PM2.5)
- Line chart: pollutant trend over time for a selected city
- Alert table: cities where PM2.5 > 25 µg/m³ (WHO limit)
- Filters: country, pollutant type, date range

Stretch Goals *(optional, bonus marks)*

- Add Open-Meteo weather API to correlate pollution with temperature/humidity
- Add a choropleth map using Folium or Plotly

Project 2 — Movie Industry Analytics

Domain: Entertainment Analytics

API	TMDB — The Movie Database (`themoviedb.org/documentation/api`)
Authentication	Free API key (register at themoviedb.org)
Movies endpoint	`https://api.themoviedb.org/3/discover/movie?api_key=YOUR_KEY`
Genres endpoint	`https://api.themoviedb.org/3/genre/movie/list?api_key=YOUR_KEY`
Docs	https://developer.themoviedb.org/docs

Description: Ingest movie metadata, ratings, and genre data from TMDB. Analyse box office trends, genre popularity over decades, and the relationship between budget and revenue.

Required Raw Tables

Table	Key Columns
`movies`	`id`, `title`, `release_date`, `genre_ids`, `budget`, `revenue`, `vote_average`, `vote_count`, `popularity`
`genres`	`id`, `name`

Required dbt Models

- **Staging:** `stg_movies` — cast `release_date` to date, extract `release_year`, filter `budget > 0` and `revenue > 0`, calculate `roi = (revenue - budget) / budget`
- **Mart:** `genre_decade_summary` — join movies to genres, group by genre + decade, compute avg rating, avg ROI, total films

Required dbt Tests (minimum 3)

- `not_null` on `movie_id`, `title`, `release_year`

- `accepted_values`: `vote_average` between 0 and 10
- `unique` on `movie_id`

Required Dashboard Components

- KPI row: total movies loaded, avg rating, highest grossing film
- Bar chart: top 10 genres by average ROI
- Line chart: average rating by release year (trend over decades)
- Scatter plot: budget vs revenue (with genre colour coding)
- Filters: genre, decade, minimum vote count

Stretch Goals *(optional, bonus marks)*

- Add TMDB Credits API to analyse top-performing directors
- Add revenue vs rating correlation analysis

Project 3 — Cryptocurrency Market Pipeline

Domain: Financial Analytics

API	CoinGecko (`api.coingecko.com`)
Authentication	None required
Markets endpoint	`https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_page=50`
History endpoint	`https://api.coingecko.com/api/v3/coins/{id}/market_chart?vs_currency=usd&days=90`
Docs	https://docs.coingecko.com

Description: Ingest historical price, volume, and market cap data for the top 50 cryptocurrencies. Build a pipeline that tracks daily market movements, volatility, and dominance shifts.

Required Raw Tables

Table	Key Columns
`coins`	`id`, `symbol`, `name`, `current_price`, `market_cap`, `total_volume`, `price_change_24h`, `last_updated`
`coin_history`	`coin_id`, `date`, `price`, `market_cap`, `total_volume`

Required dbt Models

- **Staging:** `stg_coins` — cast `last_updated` to timestamp, calculate `market_cap_rank`, filter out coins with null price
- **Mart:** `daily_market_summary` — group by date, compute total market cap, top coin by volume, avg price change across top 10

Required dbt Tests (minimum 3)

- `not_null` on `coin_id`, `symbol`, `current_price`
- Custom test: `current_price > 0`
- `unique` on `coin_id + last_updated`

Required Dashboard Components

- KPI row: total market cap, 24h volume, number of coins tracked
- Bar chart: top 10 coins by market cap
- Line chart: price trend over time for a selected coin
- Table: biggest 24h gainers and losers
- Filters: top N coins, date range

Stretch Goals *(optional, bonus marks)*

- Calculate 7-day rolling average price using dbt window functions
- Add volatility score (std dev of daily price change)

Project 4 — Open Library Book Trends

Domain: Publishing & Culture Analytics

API	Open Library (`openlibrary.org/developers/api`)
Authentication	None required
Search endpoint	`https://openlibrary.org/search.json?subject={subject}&limit=100`
Authors endpoint	`https://openlibrary.org/authors/{key}.json`
Docs	https://openlibrary.org/developers/api

Description: Ingest book metadata from Open Library across multiple subjects. Analyse publishing trends by decade, subject popularity, and most prolific authors.

Required Raw Tables

Table	Key Columns
`books`	`key`, `title`, `author_name`, `first_publish_year`, `subject`, `number_of_pages`, `edition_count`
`authors`	`key`, `name`, `birth_date`, `top_subjects`

Required dbt Models

- **Staging:** `stg_books` — cast `first_publish_year` to integer, extract `decade = (first_publish_year / 10) * 10`, filter year between 1900 and 2024
- **Mart:** `subject_decade_summary` — group by subject + decade, count books, avg pages, count unique authors

Required dbt Tests (minimum 3)

- `not_null` on `book_key`, `title`, `first_publish_year`
- Custom test: `first_publish_year between 1800 and 2024`
- `unique` on `book_key`

Required Dashboard Components

- KPI row: total books loaded, subjects covered, year range
- Bar chart: top 10 most published subjects
- Line chart: number of books published per decade (trend)
- Table: top 10 most prolific authors by book count
- Filters: subject, decade range

Stretch Goals *(optional, bonus marks)*

- Add a word cloud of most common subjects
- Compare two subjects side by side

Project 5 — GitHub Open Source Pulse

Domain: Software Engineering Analytics

API	GitHub REST API (`api.github.com`)
Authentication	GitHub Personal Access Token (free)
Issues endpoint	`https://api.github.com/repos/{owner}/{repo}/issues?state=all&per_page=100`
Repos endpoint	`https://api.github.com/repos/{owner}/{repo}`
Docs	https://docs.github.com/en/rest

Description: Ingest repository statistics, issue activity, and contributor data across a set of popular open source projects. Analyse community health, issue resolution time, and contribution patterns.

Required Raw Tables

Table	Key Columns
`issues`	`id`, `repo`, `number`, `title`, `state`, `author_login`, `created_at`, `closed_at`, `labels`
`repos`	`id`, `name`, `full_name`, `stars`, `forks`, `language`, `open_issues_count`, `created_at`

Required dbt Models

- **Staging:** `stg_issues` — cast `created_at`/`closed_at` to timestamp, calculate `days_open`, add `is_closed` boolean
- **Mart:** `repo_issue_summary` — group by repo, compute total issues, open vs closed count, avg days to close, top 5 authors

Required dbt Tests (minimum 3)

- `not_null` on `issue_id`, `repo`, `state`
- `accepted_values` on `state`: `['open','closed']`
- `unique` on `issue_id + repo`

Required Dashboard Components

- KPI row: total repos tracked, total issues, avg resolution time
- Bar chart: issues opened vs closed per repo
- Line chart: issues opened per week over time
- Table: top contributors by issues opened per repo
- Filters: repo, state (open/closed), date range

Stretch Goals *(optional, bonus marks)*

- Compare 3 repos side by side
- Add label breakdown (bug vs feature vs question)

Project 6 — World University Rankings Pipeline

Domain: Education Analytics

API	REST Countries (`restcountries.com`) + Universities API (`universities.hipolabs.com`)
Authentication	None required
Countries endpoint	`https://restcountries.com/v3.1/all?fields=name,cca2,region,subregion,population,area,capital`
Universities endpoint	`http://universities.hipolabs.com/search?country={country_name}`
Docs	https://restcountries.com / https://github.com/Hipo/university-domains-list

Description: Ingest university data by country and combine with country-level indicators (population, region). Build a pipeline that analyses the global distribution of higher education institutions.

Required Raw Tables

Table	Key Columns
`universities`	`name`, `country`, `alpha_two_code`, `state_province`, `web_pages`, `domains`
`countries`	`name`, `alpha2_code`, `region`, `subregion`, `population`, `area`, `capital`

Required dbt Models

- **Staging:** `stg_universities` — clean nulls, standardise country name, join `alpha_two_code` to countries for region/subregion enrichment

- **Mart:** `country_university_summary` — group by country + region, count universities, compute `universities_per_million`

Required dbt Tests (minimum 3)

- `not_null` on `university_name`, `country`
- `not_null` on `region` after join
- Custom test: `universities_per_million > 0`

Required Dashboard Components

- KPI row: total universities, countries covered, regions represented
- Bar chart: top 20 countries by number of universities
- Bar chart: universities per million population by region
- Table: country-level breakdown with region and count
- Filters: region, subregion, minimum university count

Stretch Goals *(optional, bonus marks)*

- Add a choropleth map using Plotly Express
- Normalise by population to find most 'university-dense' countries

---

7. Academic Integrity

You are expected to use AI tools (ChatGPT, Claude, GitHub Copilot) as **learning aids** — not as a substitute for understanding. The following rules apply:

- You must be able to **explain every line of your code** during the Q&A. If you cannot explain it, it will not receive marks.
- The report must be written in your own words. AI-generated reports are detectable and will be reviewed with Turnitin.
- All group members are **equally responsible** for all parts of the project. "I didn't do that part" is not accepted during Q&A.
- Code similarity between groups will be checked. Identical or near-identical pipelines will result in **zero marks for both groups**.
- The most effective way to prepare for Q&A is to build the project yourself, step by step. Debugging your own errors is the best learning.

---

8. Recommended Timeline

Week	Dates	Recommended Progress
Week 8	16 Jun	Project kick-off — groups announced
Week 9	23 Jun	API keys registered, `pipeline.py` working, raw tables loaded into PostgreSQL
Week 10	30 Jun	dbt staging model complete, `dbt run` passing

Week	Dates	Recommended Progress
Week 11	7 Jul	dbt mart model + all tests passing, Streamlit prototype running
Week 12	14 Jul	Dashboard polished, report complete, code reviewed and cleaned, README written
Week 13	**21 Jul**	**SUBMISSION DEADLINE + PRESENTATIONS — report PDF + GitHub link via Blackboard + live demo + Q&A in class**

Questions? Contact Dr. Maira Kotsovoulou via email or during office hours. Good luck!